

UNIVERZITET U NIŠU

ELEKTRONSKI FAKULTET

SEMINARSKI RAD

Ekstrakcija i analiza aspekata proizvoda iz korisničkih recenzija pomoću *LangExtract* biblioteke

-WEB MINING-

Mentor:
prof. dr Miloš Bogdanović

Student:
Jovan Paunović 2122

Niš, april 2026. godina

SADRŽAJ

1. UVOD 4
2. *WEB MINING* 5
   2.1. Vrste *web mining*-a 5
   2.2. Proces *web mining*-a i primene 6
3. ANALIZA SENTIMENTA 7
   3.1. Pristupi analizi sentimenta 7
   3.2. Vrste analize sentimenta 8
   3.3. Analiza sentimenta na nivou aspekta 8
   3.4. Izazovi analize sentimenta 9
4. EKSTRAKCIJA INFORMACIJA 10
   4.1. Pristupi ekstrakciji informacija 10
   4.2. *LangExtract* biblioteka 11
      4.2.1. Koncept i način rada 12
      4.2.2. Definisanje zadatka ekstrakcije 13
5. KLASTERIZACIJA TEKSTA 15
   5.1. Predstavljanje teksta u vektorskom prostoru 15
      5.1.1. Klasične metode reprezentacije 16
      5.1.2. *Word Embeddings* 16
      5.1.3. *Sentence Transformers* 17
      5.1.4. Poređenje tehnika reprezentacije teksta 17
   5.2. Algoritmi klasterizacije 18
   5.3. *Agglomerative Clustering* 19
      5.3.1. Dendrogrami 20
      5.3.2. Strategije spajanja 20
      5.3.3. Prednosti i mane 21
   5.4. Metrike kvaliteta klasterizacije 21
6. PRAKTIČNI DEO 23
   6.1. Opis sistema i arhitektura 23
   6.2. Skup podataka 24
   6.3. Struktura baza podataka 25
   6.4. Ekstrakcija karakteristika 26

6.4.1. Definisanje zadatka ekstrakcije....26
6.4.2. Paralelna obrada recenzija....27
6.4.3. Primer ekstrakcije....27
6.5. Vektorske reprezentacije i klasterizacija....28
6.5.1. Generisanje vektorskih reprezentacija fraza....28
6.5.2. Konfiguracija klasterizacije....28
6.6. Imenovanje klastera i agregacija sentimenta....29
6.6.1. Imenovanje klastera....29
6.6.2. Agregacija sentimenta....30
6.7. Rezultati i evaluacija....30
6.7.1. Primer analize konkretnog uređaja....31
6.7.2. Diskusija kvaliteta klasterizacije....31
6.7.3. Ograničenja sistema....32
7. ZAKLJUČAK....33
LITERATURA....34

1. UVOD

U savremenom digitalnom okruženju, korisnici svakodnevno ostavljaju veliki broj recenzija proizvoda na platformama kao što su *Amazon*, *Google Shopping* itd. Ove recenzije predstavljaju bogat izvor informacija – kako za potencijalne kupce, tako i za proizvođače koji žele da razumeju jake i slabe strane svojih proizvoda. Međutim, ručna analiza stotina ili hiljada recenzija nije praktična, što nameće potrebu za automatizovanim pristupima obradi prirodnog jezika.

Tradicionalne metode analize sentimenta uglavnom rade na nivou celog dokumenta – određuju da li je recenzija u celini pozitivna ili negativna. Takav pristup gubi važne informacije – u kontekstu recenzija mobilnih telefona, korisnik može biti zadovoljan kamerom, ali nezadovoljan performansama telefona. Analiza sentimenta na nivou aspekata (*Aspect-Based Sentiment Analysis – ABSA*) rešava ovaj problem tako što identifikuje konkretne karakteristike proizvoda i određuje sentiment vezan za svaku od njih posebno.

Cilj ovog rada je projektovanje i implementacija sistema koji automatski ekstrahuje aspekte proizvoda iz korisničkih recenzija i analizira sentiment vezan za svaki aspekt. Sistem koristi *LangExtract* biblioteku za ekstrakciju strukturiranih informacija iz teksta pomoću velikih jezičkih modela, tehnike vektorskih reprezentacija izvučenih fraza u semantičkom prostoru, kao i klasterizaciju za grupisanje semantički sličnih aspekata u smislene kategorije.

Rad je organizovan na sledeći način: najpre se uvodi pojam *Web Mining*-a i njegova uloga u analizi *online* sadržaja, nakon čega sledi opis analize sentimenta, sa posebnim osvrtom na analizu na nivou aspekata. Zatim se opisuje ekstrakcija informacija i *LangExtract* biblioteka, kao i vektorske reprezentacije teksta i semantička sličnost. Posebna pažnja posvećena je algoritmima klasterizacije, sa fokusom na *Agglomerative Clustering*. Na kraju rada prikazuje se praktična implementacija sistema – od arhitekture i pripreme podataka do ekstrakcije, klasterizacije i evaluacije rezultata.

2. *WEB MINING*

Web mining predstavlja primenu tehnika *data mining*-a, mašinskog učenja i analitičkih metoda u cilju pronalaženja smislenih obrazaca i znanja iz ogromne količine podataka dostupnih na *web*-u. Osnovna svrha *web mining*-a je transformacija sirovog, nestrukturiranog *web* sadržaja u upotrebljive informacije pogodne za dalju analizu i odlučivanje. Za razliku od klasičnog *data mining*-a koji pretežno radi sa strukturiranim skupovima podataka organizovanim u tabele i zapise, *web mining* se suočava sa heterogenim, dinamičkim i često neurednim podacima – tekstom, slikama, multimedijom, hiperlinkovima i serverskim logovima. Iz tog razloga, *web mining* kombinuje koncepte iz više oblasti kao što su *data mining*, obrada prirodnog jezika, pronalaženje informacija i veštačka inteligencija [1].

2.1. Vrste *web mining*-a

Web mining se deli na tri osnovne kategorije, u zavisnosti od tipa podataka koji se analiziraju. Na slici 2.1. je prikazana ta podela.



Slika 2.1. Podela Web mining-a na tri kategorije [1]

Analiza sadržaja (engl. *web content mining*) bavi se ekstrakcijom korisnih informacija iz stvarnog sadržaja *web* stranica – teksta, slika, audio i video materijala, kao i metapodataka. Koristi tehnike obrade prirodnog jezika, klasifikacije i klasterizacije kako bi pretvorila nestrukturirani ili polustrukturirani sadržaj u organizovanu, pretraživu formu. Primenjuje se za poboljšanje relevantnosti pretrage, organizaciju sadržaja i pronalaženje informacija.

Analiza korišćenja (engl. *web usage mining*) bavi se ponašanjem korisnika na osnovu serverskih logova, tokova klikova, kolačića i podataka o sesijama. Cilj je otkrivanje obrazaca navigacije, korisničkih preferencija i trendova u načinu korišćenja *web* servisa. Na osnovu ovih saznanja mogu se graditi sistemi za personalizaciju, preporuku sadržaja i detekciju prevara.

Analiza strukture (engl. *web structure mining*) bavi se strukturom linkova na *web*-u kako bi se otkrili odnosi između stranica i razumeo način na koji su informacije međusobno povezane. *Web* se posmatra kao usmereni graf u kome su čvorovi stranice, a grane hiperlinkovi. Ovaj pristup omogućava identifikaciju uticajnih stranica, otkrivanje zajednica i navigacionih putanja unutar sajtova, kao i optimizaciju rangiranja i dizajna *web* sajtova.

2.2. Proces *web mining*-a i primene

Bez obzira na vrstu, proces *web mining*-a odvija se kroz nekoliko faza. U fazi prikupljanja podataka sakuplja se sirovi materijal sa *web* stranica, logova, metapodataka i hiperlinkova. Zatim sledi predobrada, koja podrazumeva uklanjanje šuma, parsiranje HTML strukture, obradu nedostajućih vrednosti i konverziju podataka u format pogodan za analizu. U fazi otkrivanja obrazaca primenjuju se odgovarajuće tehnike – klasterizacija, klasifikacija, analiza asocijacija ili sekvencijalna analiza. Na kraju, otkriveni obrasci se interpretiraju i koriste za donošenje odluka.

Web mining nalazi primenu u brojnim oblastima. Koristi se za personalizovani marketing i optimizaciju *e-commerce* platformi, poboljšanje indeksiranja i rangiranja u pretraživačima, detekciju prevara na osnovu anomalija u obrascima pregledanja, kao i za analizu sentimenta – ekstrakciju mišljenja i emocija iz recenzija, komentara i objava na društvenim mrežama i *web*-u. Upravo ova poslednja primena predstavlja polaznu tačku ovog rada.

3. ANALIZA SENTIMENTA

Analiza sentimenta predstavlja oblast obrade prirodnog jezika (engl. *Natural Language Processing – NLP*) koja se bavi automatskim prepoznavanjem i ekstrakcijom subjektivnih informacija iz teksta – stavova, mišljenja i emocija koje autori izražavaju prema određenim entitetima, proizvodima ili uslugama. Dok tradicionalne metode pretrage i analize teksta uglavnom rade sa objektivnim, činjeničnim sadržajem, analiza sentimenta fokusira se upravo na subjektivnu dimenziju jezika.

U savremenom digitalnom okruženju, korisnici svakodnevno ostavljaju ogromne količine tekstualnih podataka – recenzije proizvoda, komentare na društvenim mrežama, odgovore na ankete, poruke korisničke podrške. Ručna obrada podataka ovog obima nije izvodljiva, što čini automatizovanu analizu sentimenta izuzetno vrednom. Kompanije je koriste za praćenje reputacije brenda, razumevanje korisničkih iskustava, unapređenje proizvoda i donošenje poslovnih odluka zasnovanih na podacima [2], [3].

U ovom poglavlju opisuju se osnovni pristupi analizi sentimenta, nivoi na kojima se ona može vršiti, kao i analiza sentimenta na nivou aspekta koja predstavlja tehniku na kojoj se zasniva ovaj rad. Na kraju su istaknuti i glavni izazovi sa kojima se automatizovane metode susreću pri obradi prirodnog jezika.

3.1. Pristupi analizi sentimenta

Postoje tri osnovna pristupa analizi sentimenta: pristup zasnovan na pravilima, pristup zasnovan na mašinskom učenju i hibridni pristup.

Pristup zasnovan na pravilima oslanja se na unapred definisane rečnike reči sa pridruženim sentiment vrednostima – pozitivnim, negativnim ili neutralnim – uz skup lingvističkih pravila koja uzimaju u obzir modifikatore poput negacije ili pojačivača. Na primer, reč “loš” nosi negativnu vrednost, ali uz pojačivač “veoma” ta vrednost se amplifikuje, dok negacija “nije loš” menja polaritet. Prednost ovog pristupa je interpretabilnost i brzina, ali je njegova primena ograničena jer se teško nosi sa suptilnostima jezika, sarkazmom i novim izrazima koji nisu obuhvaćeni rečnikom.

Pristup zasnovan na mašinskom učenju koristi algoritme koji uče obrasce iz anotiranih skupova podataka. Klasični modeli poput *Naïve Bayes*, logističke regresije i SVM (engl. *Support Vector Machine*) dugo su bili dominantni u ovoj oblasti. Sa razvojem dubokog učenja, na scenu su stupile rekurentne neuronske mreže i *transformer* modeli, koji su sposobni da razumeju kontekst i semantiku na znatno višem nivou. Upravo u ovu kategoriju spadaju veliki jezički modeli koji se koriste u sistemu opisanom u ovom radu.

Hibridni pristup kombinuje oba navedena načina – lingvistička pravila obezbeđuju interpretabilnost i pouzdanost u jasnim slučajevima, dok modeli mašinskog učenja pokrivaju složenije i dvosmislene situacije. Ovaj pristup često daje najbolje rezultate, ali zahteva i najviše resursa za razvoj i održavanje.

3.2. Vrste analize sentimenta

Postoji nekoliko vrsta analize sentimenta koje se razlikuju po tome šta tačno mere i kakav izlaz proizvode [3].

Fino granulisana analiza sentimenta (engl. *fine-grained*) proširuje osnovnu klasifikaciju na 3 klase (pozitivno, negativno, neutralno) uvođenjem finijih kategorija – na primer, veoma pozitivno, pozitivno, neutralno, negativno i veoma negativno. Ovakav pristup omogućava preciznije merenje intenziteta sentimenta, slično načinu na koji sajtovi za recenzije koriste sistem ocenjivanja zvezdicama. Koristan je kada je važno razlikovati ne samo smer, već i jačinu izraženog stava.

Analiza sentimenta zasnovana na detekciji emocija prevazilazi koncept polariteta i pokušava da identifikuje konkretna emocionalna stanja iza teksta – sreću, tugu, ljutnju, iznenađenje, strah ili gađenje. Umesto da odredi samo da li je tekst negativan, ovaj pristup može prepoznati da li korisnik izražava frustraciju, razočaranje ili bes. Zbog složenosti i subjektivnosti emocija, ovo je tehnički najzahtevnija vrsta analize.

Analiza zasnovana na nameri (engl. *intent-based*) usmerena je na prepoznavanje korisnikove namere iza teksta – da li korisnik planira kupovinu, razmatra odustajanje od usluge, traži podršku ili izražava interesovanje za nadogradnju. Ova vrsta analize posebno je korisna u sistemima za korisničku podršku i *e-commerce* platformama.

Analiza sentimenta na nivou aspekta (ABSA) identifikuje konkretne karakteristike ili aspekte proizvoda i usluga koji se pominju u tekstu i za svaki od njih zasebno određuje sentiment. Za razliku od prethodno navedenih vrsta koje posmatraju tekst kao celinu, ABSA pruža granularni uvid u korisničko iskustvo – recenzija mobilnog telefona može istovremeno izražavati pozitivan stav prema kameri i negativan prema performansama. Upravo ova sposobnost čini ovu vrstu analize najkorisnijom vrstom analize za potrebe ovog rada, pa će biti detaljnije obrađena u narednoj sekciji.

3.3. Analiza sentimenta na nivou aspekta

Kao što je navedeno, ABSA identifikuje aspekte i za svaki određuje sentiment zasebno. Centralni pojam je upravo aspekt – karakteristika ili atribut entiteta o kome se govori u tekstu. U kontekstu recenzija mobilnih telefona, aspekti mogu biti kamera, baterija, ekran, cena, dizajn i slično. Identifikacija aspekata može se vršiti na različite načine: pristupom zasnovanim na pravilima i rečnicima, statističkim metodama ili, sve češće, primenom velikih jezičkih modela koji su sposobni da prepoznaju aspekte i u implicitnom obliku, bez njihovog eksplicitnog navođenja u tekstu [4].

ABSA zadatak se u literaturi najčešće formuliše kao kombinacija dva podzadatka. Prvi je ekstrakcija termina aspekata (engl. *Aspect Term Extraction – ATE*) – pronalaženje reči ili fraza u tekstu koje označavaju aspekte. Drugi je klasifikacija sentimenta aspekta (*Aspect Term Sentiment Classification – ATSC*) – određivanje polariteta (pozitivan, negativan, neutralan) za

svaki identifikovani aspekt u datom kontekstu. U praksi se ova dva zadatka često spajaju u jedan, gde se aspekti i njihovi polariteti određuju istovremeno.

Sa pojavom pretreniranih *transformer* modela i velikih jezičkih modela, tačnost ABSA sistema značajno je porasla, tako da oni sada ostvaruju rezultate uporedive sa specijalizovanim modelima čak i bez dodatnog treniranja, a uz fino podešavanje postižu vrhunske performanse na standardnim *benchmark* skupovima podataka. Pored toga, detaljna formulacija upita (engl. *prompt engineering*) pokazala se kao ključan faktor za kvalitet rezultata u scenarijima bez dodatnog treniranja [5].

U ovom radu, ABSA služi kao osnova sistema koji automatski ekstrahuje aspekte mobilnih telefona iz korisničkih recenzija i za svaki aspekt određuje koliko puta je pominjan pozitivno, a koliko puta negativno, čime se dobija detaljan profil percepcije svakog uređaja.

3.4. Izazovi analize sentimenta

Uprkos značajnom napretku u oblasti, automatizovana analiza sentimenta i dalje se suočava sa nizom izazova koji proizilaze iz prirodne složenosti ljudskog jezika.

Jedan od najčešćih problema je negacija – upotreba negativnih reči koja obrće značenje iskaza. Rečenica poput "nisu me razočarali" zapravo izražava pozitivan stav, ali jednostavniji modeli zasnovani na rečnicima reč "razočarali" mogu pogrešno klasifikovati kao negativan signal. Slično važi i za pojačivače i umanjivače – "malo loš" i "izuzetno loš" nose isti polaritet, ali sasvim različit intenzitet.

Sarkazam i ironija predstavljaju još veći izazov, jer podrazumevaju upotrebu pozitivnih reči u negativnom kontekstu. Iskaz poput "Odlično, baterija je ponovo prazna za sat vremena" sadrži reč pozitivnog značenja, ali izražava jasno nezadovoljstvo. Prepoznavanje ovakvog sadržaja zahteva razumevanje šireg konteksta, što je izvan dometa jednostavnih pristupa.

Problem konteksta javlja se i u situacijama gde isti izraz nosi različit sentiment u zavisnosti od postavljenog pitanja ili teme razgovora. Isti odgovor korisnika može biti pozitivan u jednom kontekstu i negativan u drugom, što sistem mora uzeti u obzir pri klasifikaciji.

Idiomatski izrazi i kolokvijalni jezik dodatno otežavaju analizu, jer njihovo doslovno tumačenje često ne odgovara stvarnom značenju. Ovaj problem posebno dolazi do izražaja u korisničkim recenzijama, koje su pisane neformalno i obiluju žargonom, skraćenicama i nestandardnim formulacijama.

Upravo ovi izazovi opravdavaju primenu velikih jezičkih modela u sistemu opisanom u ovom radu. Za razliku od pristupa zasnovanih na pravilima ili klasičnih modela mašinskog učenja, LLM-ovi su sposobni da razumeju kontekst, prepoznaju implicitno značenje i izađu na kraj sa suptilnostima prirodnog jezika na način koji je znatno bliži ljudskom razumevanju teksta [2].

4. EKSTRAKCIJA INFORMACIJA

Ekstrakcija informacija (engl. *Information Extraction – IE*) je automatizovani proces prevođenja nestrukturiranog ili polustrukturiranog teksta u strukturirani, mašinski čitljiv oblik. Algoritmi za ekstrakciju informacija prepoznaju i izdvajaju relevantne entitete, relacije, događaje i sentiment iz teksta, a zatim ih klasifikuju i čuvaju u formatu pogodnom za pretraživanje i dalju analizu [6].

Važno je razlikovati ekstrakciju informacija od pukog pretraživanja teksta. Sistemi za pronalaženje informacija (engl. *Information Retrieval*) pronalaze dokumente koji su relevantni za zadati upit – ali sadržaj tih dokumenata ostaje nestrukturiran. Ekstrakcija informacija ide korak dalje: iz pronađenog teksta izvlači konkretne podatke i organizuje ih u strukturirani oblik sa jasno definisanim atributima.

Ekstrakcija informacija obuhvata nekoliko međusobno povezanih zadataka. Prepoznavanje imenovanih entiteta (engl. *Named Entity Recognition – NER*) bavi se identifikacijom i klasifikacijom specifičnih izraza u tekstu – osoba, organizacija, lokacija, proizvoda i sličnih kategorija. Ekstrakcija relacija (engl. *Relation Extraction*) određuje semantičke veze između identifikovanih entiteta, dok ekstrakcija događaja prepoznaje konkretna dešavanja u tekstu zajedno sa njihovim učesnicima, vremenom i mestom. Razrešavanje koreferencija (engl. *Coreference Resolution*) identifikuje sve izraze u tekstu koji se odnose na isti entitet, čime se otklanja dvosmislenost. Konačno, popunjavanje šablona (engl. *Template Filling*) podrazumeva mapiranje izvučenih informacija u unapred definisane strukture sa fiksnim poljima – što je konceptualno najbliže pristupu koji se koristi u ovom radu [7].

U okviru NLP ekosistema, ekstrakcija informacija predstavlja jedan od temeljnih zadataka koji omogućava niz naprednijih primena – od sistema za odgovaranje na pitanja i izgradnje baza znanja, do analize sentimenta i automatskog sažimanja teksta. U kontekstu ovog rada, posebno je relevantan zadatak popunjavanja šablona: sistem prima unapred definisanu šemu koja opisuje koje karakteristike proizvoda treba izvući iz recenzije, a jezički model tu šemu popunjava na osnovu sadržaja teksta. Na taj način ekstrakcija informacija postaje direktna osnova za analizu sentimenta na nivou aspekta, jer bez identifikacije aspekata nema ni sentimenta koji bi im se pripisao.

4.1. Pristupi ekstrakciji informacija

Tehnike ekstrakcije informacija razvijale su se postepeno, od jednostavnih pristupa zasnovanih na pravilima do savremenih modela dubokog učenja i velikih jezičkih modela [7].

Pristup zasnovan na pravilima parsira dokumente i identifikuje entitete na osnovu unapred definisanih obrazaca i definicija. Na primer, pravilo može glasiti da svaka reč koja počinje velikim slovom i nalazi se u neposrednoj blizini reči “osnovana” predstavlja naziv kompanije. Ovakvi sistemi su pouzdani i brzi u usko definisanim domenima, ali nisu skalabilni

– svaka promena u strukturi teksta ili pojava novih obrazaca zahteva ručno ažuriranje pravila, a skaliranje na različite domene je izuzetno naporno.

Pristup zasnovan na klasifikaciji predstavlja korak ka automatizaciji – umesto ručnog definisanja pravila, modeli mašinskog učenja se treniraju na označenim skupovima podataka i uče da prepoznaju obrasce između entiteta i njihovih atributa. Jednom istreniran model može predviđati oznake za entitete u novim, neviđenim tekstovima. Ovaj pristup je fleksibilniji od pristupa zasnovanog na pravilima, ali i dalje zahteva značajnu količinu kvalitetno anotiranih podataka i teže se prilagođava domenima koji nisu zastupljeni u skupu za trening.

Označavanje sekvenci (engl. *Sequence Labeling*) predstavlja temelj savremenog NLP-a i koristi modele dubokog učenja za identifikaciju i označavanje komponenti ulaznog niza. Za razliku od prethodnih pristupa, označavanje sekvenci hvata i zavisnosti između delova ulaznog niza, što znači da model razume kako kontekst jedne reči utiče na interpretaciju druge. *Transformer* modeli posebno su efikasni u ovom zadatku zahvaljujući mehanizmu pažnje (engl. *attention mechanism*) koji im omogućava da prate kontekstualne veze na velikim rastojanjima unutar teksta.

Upravo iz *transformer* arhitekture nastali su **veliki jezički modeli** koji danas predstavljaju najznačajniji pravac u ekstrakciji informacija. Oni donose nekoliko ključnih prednosti u odnosu na prethodne pristupe. Zahvaljujući pretreniranju na ogromnim tekstualnim korpusima, sposobni su da vrše ekstrakciju i bez domenski specifičnog treniranja – u tzv. *zero-shot* ili *few-shot* režimu, gde je dovoljno opisati zadatak u vidu upita i opciono dati neke primere. Pored toga, LLM-ovi mogu da generišu strukturirani izlaz u skladu sa unapred zadatom JSON šemom, što znači da ekstrakcija i strukturiranje podataka mogu biti obavljeni u jednom koraku. Ova mogućnost je posebno vredna pri radu sa korisničkim recenzijama, koje su po prirodi neformalne, raznolike i teško podložne striktnim pravilima.

4.2. *LangExtract* biblioteka

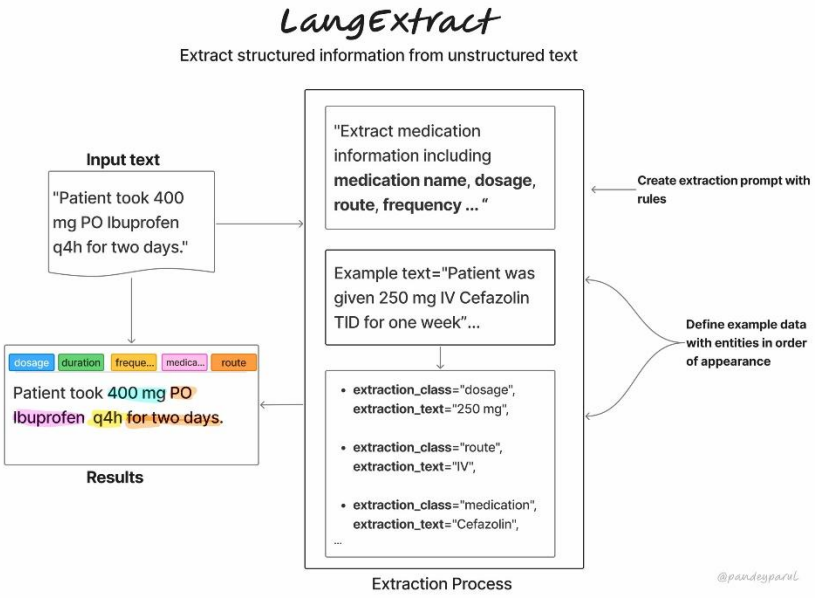
LangExtract predstavlja *Python* biblioteku otvorenog koda, razvijenu od strane kompanije *Google*, koja je dizajnirana da poveže napredne mogućnosti rezonovanja velikih jezičkih modela sa inženjerskom potrebom za pouzdanim, strukturiranim podacima. U svojoj osnovi, ova biblioteka služi za automatizovanu ekstrakciju i organizaciju informacija iz nestrukturiranih tekstualnih dokumenata, oslanjajući se na korisnički definisana uputstva. Za razliku od tradicionalnog, ručnog pisanja upita – gde je izlaz modela često sklon halucinacijama, varijacijama u formatu i gde zahteva kompleksno naknadno parsiranje – *LangExtract* uvodi sistemski pristup koji garantuje doslednost i konzistentnost rezultata [8]. Zvanična dokumentacija biblioteke posebno ističe nekoliko ključnih razloga za njenu upotrebu [9]:

- **Precizno označavanje izvora informacije** (engl. *source grounding*) – biblioteka mapira svaku izvučenu informaciju na njenu tačnu poziciju u izvornom tekstu. Ovo omogućava vizuelno isticanje podataka, olakšavajući ljudsku verifikaciju i obezbeđujući potpunu transparentnost procesa.

- **Pouzdani strukturirani izlazi** – korišćenjem tehnika kontrolisanog generisanja kod podržanih modela, *LangExtract* striktno nameće izlaznu šemu definisanu kroz primere, čime se eliminiše nepredvidivost slobodnog LLM odgovora.
- **Optimizacija za dugačke dokumente –** biblioteka uspešno prevazilazi problem pada pažnje kod modela na dugačkim tekstovima (efekat "igle u plastu sena") primenom pametnog deljenja teksta na segmente (engl. *chunking*), paralelne obrade i višestrukih prolaza u cilju postizanja maksimalnog odziva.
- **Interaktivna vizuelizacija –** izlazni rezultati se mogu automatski transformisati u interaktivne HTML izveštaje, omogućavajući istraživačima i inženjerima da intuitivno pregledaju hiljade ekstrahovanih entiteta unutar njihovog originalnog konteksta.
- **Domenska prilagodljivost** – sistem se fleksibilno prilagođava specifičnim zadacima i zahtevima različitih domena isključivo kroz inženjering upita i kvalitetne primere, uspešno koristeći ugrađeno znanje modela o svetu, bez potrebe za skupim i kompleksnim procesom finog podešavanja modela (engl. *fine-tuning*).
- **Fleksibilna LLM podrška** – biblioteka podržava *cloud* modele poput *Gemini* ili *GPT* modela, kao i lokalne modele otvorenog koda preko *Ollama* interfejsa.

4.2.1. Koncept i način rada

Rad sa *LangExtract* bibliotekom svodi se na tri koraka. Najpre se definiše zadatak ekstrakcije kroz upit koji jasno opisuje šta treba izvući iz teksta. Zatim se pruža jedan ili više visokokvalitetnih primera koji pokazuju model očekivanog izlaza. Na kraju se prosleđuje ulazni tekst i pokreće ekstrakcija – biblioteka interno formuliše upit prema LLM-u, dobija strukturirani odgovor i mapira svaku izvučenu stavku nazad na tačnu poziciju u originalnom tekstu [8]. Na slici 4.1. je prikazan tok rada *LangExtract* biblioteke.



Slika 4.1. Tok rada LangExtract biblioteke [8]

Upit predstavlja srž zadatka ekstrakcije – on opisuje koje informacije treba izvući, u kom obliku i na osnovu kojih kriterijuma. Dobro formulisan upit je koncizan, nedvosmislen i eksplicitan u pogledu toga šta se smatra relevantnom informacijom. Na primer, upit može nalagati da se ekstrahuju karakteristike proizvoda i sentiment koji je prema njima izražen, da se koriste tačni izrazi iz originalnog teksta bez parafraziranja, i da se stavke navode u redosledu pojavljivanja. Preciznost formulacije direktno se odražava na kvalitet rezultata – nejasan upit vodi ka nedoslednim ili nepotpunim ekstrakcijama.

Primeri igraju podjednako važnu ulogu kao i upit. Kroz primere model uči konkretan format izlaza, nivo granularnosti koji se očekuje i način na koji se granični slučajevi tretiraju. Kvalitet primera direktno određuje kvalitet ekstrakcije – jedan dobro odabran primer često donosi bolje rezultate od mnoštva površnih. Upravo iz tog razloga korisnik ostaje važan deo procesa: dok LLM obavlja teški deo posla, odgovornost za definisanje zadatka i formulisanje primera ostaje na korisniku.

Zahvaljujući osobini biblioteke da precizno označava izvor ekstrahovane informacije moguća je vizualizacija rezultata. Na levoj strani slike 4.1. prikazani su rezultati: svaka ekstrahovana stavka obeležena je svojom klasom (*extraction_class*) i tačnim tekstom iz originalnog dokumenta (*extraction_text*), a u vizualizaciji su različite klase označene različitim bojama direktno u izvornom tekstu.

4.2.2. Definisanje zadatka ekstrakcije

Kao što je već rečeno, zadatak ekstrakcije u *LangExtract* biblioteci definiše se kroz dva elementa: upit koji opisuje šta treba izvući i primere koji pokazuju očekivani format izlaza. Oba elementa zajedno se prosleđuju funkciji *lx.extract()* koja pokreće ekstrakciju.

Upit se zadaje kao tekstualni opis zadatka kroz parametar *prompt_description*. Dobra praksa nalaže da upit bude koncizan i eksplicitan – treba da navede koje klase entiteta se ekstrahuju, da zahteva upotrebu tačnih izraza iz originalnog teksta bez parafraziranja, i da specificira redosled navođenja stavki. Nejasan ili previše uopšten upit direktno se odražava na nedoslednost rezultata.

Primeri se definišu kroz *ExampleData* objekte, od kojih svaki sadrži primer ulaznog teksta i listu očekivanih ekstrakcija. Svaka ekstrakcija je *Extraction* objekat sa sledećim poljima: *extraction_class* koji označava tip entiteta (npr. "*aspect*"), *extraction_text* koji sadrži tačan tekst iz izvora, i opcioni *attributes* rečnik koji nosi dodatne atribute poput sentimenta. Kvalitet i reprezentativnost primera ključni su za uspeh ekstrakcije – model iz njih uči format, nivo granularnosti i način tretiranja graničnih slučajeva.

Sve navedeno se objedinjuje u pozivu funkcije *lx.extract()*, kojoj se prosleđuju ulazni tekst, upit, primeri i identifikator modela. Na slici 4.2. nalazi se primer koji ilustruje kompletan zadatak ekstrakcije aspekata i sentimenta iz recenzije mobilnog telefona.



```python
import langextract as lx
import textwrap

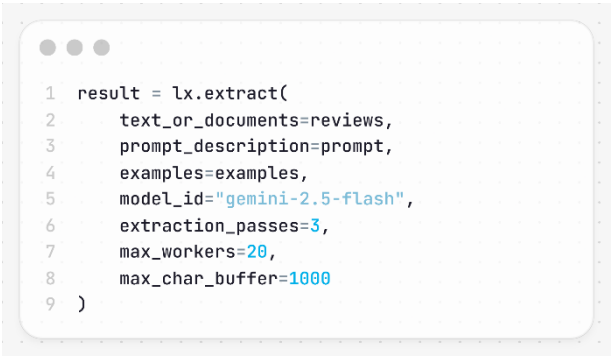
# Definisanje prompta
prompt = textwrap.dedent("""\
    Extract product aspects and their sentiment from the review.
    Use exact text spans from the original text. Do not paraphrase.
    Return entities in order of appearance.""")

# Definisanje primera
examples = [
    lx.data.ExampleData(
        text="The camera is excellent, but the battery drains too fast.",
        extractions=[
            lx.data.Extraction(
                extraction_class="aspect",
                extraction_text="camera",
                attributes={"sentiment": "positive"}
            ),
            lx.data.Extraction(
                extraction_class="aspect",
                extraction_text="battery drains too fast",
                attributes={"sentiment": "negative"}
            ),
        ]
    )
]

# Pokretanje ekstrakcije
result = lx.extract(
    text_or_documents="Screen quality is great, but the speaker is disappointing.",
    prompt_description=prompt,
    examples=examples,
    model_id="gemini-2.5-flash",
)
```

Slika 4.2. Primer kompletnog zadatka ekstrakcije aspekata i sentimenta iz recenzije

Za obradu većih skupova podataka, kao što su hiljade korisničkih recenzija, biblioteka nudi dodatne parametre koji poboljšavaju pokrivenost i efikasnost. Parametar *extraction_passes* pokreće višestruke prolaze kroz tekst, čime se povećava verovatnoća da nijedna relevantna informacija ne bude izostavljena. Parametar *max_workers* kontroliše broj paralelnih procesa i značajno ubrzava obradu. Parametar *max_char_buffer* određuje veličinu segmenata na koje se tekst deli pre prosleđivanja modelu, čime se prevazilaze ograničenja kontekstnog prozora. Pored toga, u pozivu *extract()* funkcije se definiše i identifikator modela koji će se koristiti. Za obradu velikog skupa recenzija poziv bi izgledao kao na slici 4.3.



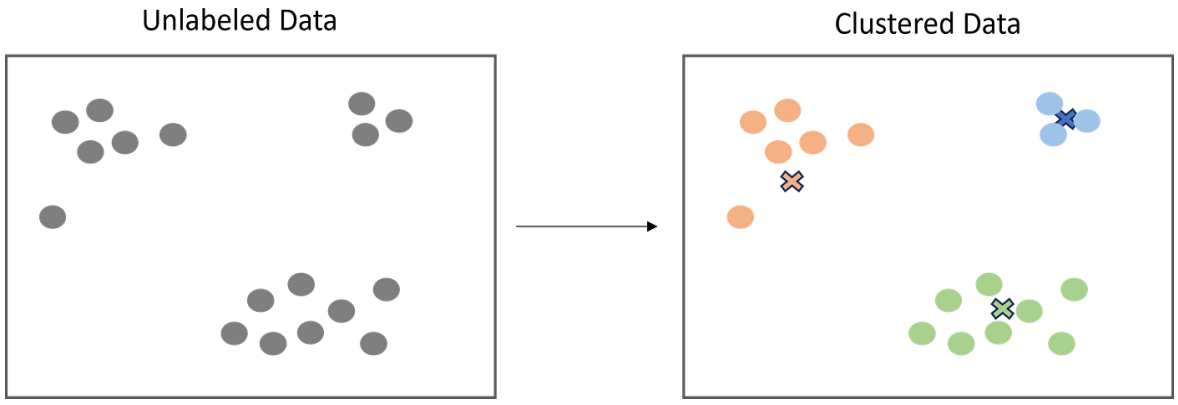
```python
result = lx.extract(
    text_or_documents=reviews,
    prompt_description=prompt,
    examples=examples,
    model_id="gemini-2.5-flash",
    extraction_passes=3,
    max_workers=20,
    max_char_buffer=1000
)
```

Slika 4.3. Primer poziva extract() funkcije

5. KLASTERIZACIJA TEKSTA

Nakon što sistem ekstrahuje aspektne fraze iz korisničkih recenzija primenom *LangExtract* biblioteke, javlja se novi problem: izvučene fraze su sirove i nekonzistentne. Različiti korisnici isti aspekt opisuju na različite načine – „baterija", "trajanje baterije", "vek baterije" i "punjenje" sve se odnose na istu karakteristiku proizvoda, ali su tekstualno potpuno različite fraze. Bez dodatne obrade, svaka od njih bila bi tretirana kao zasebna kategorija, što bi rezultiralo stotinama sitnih, fragmentiranih grupa umesto smislenih, konsolidovanih aspekata.

Rešenje ovog problema leži u klasterizaciji teksta – tehnici grupisanja tekstualnih jedinica (reči, fraza ili rečenica) na osnovu njihove semantičke sličnosti (ilustrovano na slici 5.1). Međutim, algoritmi klasterizacije ne mogu da rade direktno sa tekstom: potrebno je najpre prevesti fraze u numerički oblik koji čuva njihovo značenje. Iz tog razloga, ovo poglavlje najpre opisuje metode reprezentacije teksta u vektorskom prostoru, od klasičnih pristupa do savremenih semantičkih vektorskih reprezentacija (engl. *embeddings*), a zatim prelazi na algoritme klasterizacije i metrike za procenu njihovog kvaliteta.



Slika 5.1. Klasterizacija podataka [10]

5.1. Predstavljanje teksta u vektorskom prostoru

Da bi algoritam klasterizacije mogao da proceni koliko su dve fraze slične, mora postojati način da se ta sličnost izmeri matematički. Tekst sam po sebi nije pogodan za takva izračunavanja – potrebno ga je prevesti u vektore brojeva u kojima geometrijska blizina odgovara semantičkoj sličnosti. Ovo je fundamentalni problem reprezentacije teksta, i različiti pristupi rešavaju ga sa različitim stepenom uspešnosti [10]. Važno je napomenuti da klasične metode reprezentacije zahtevaju eksplicitnu predobradu teksta – tokenizaciju, uklanjanje stop reči i lematizaciju – dok SBERT, koji se koristi u ovom radu, ne zahteva ove korake, što eliminiše potrebu za ručnom predobradom.

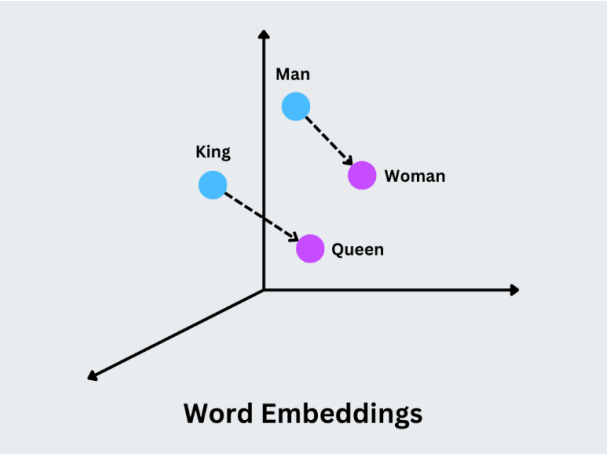
5.1.1. Klasične metode reprezentacije

Najjednostavniji pristup je model vreće reči (engl. *Bag of Words – BoW*), koji tekst predstavlja kao vektor frekvencija reči – svaka dimenzija odgovara jednoj reči iz rečnika, a vrednost označava koliko puta se ta reč pojavljuje u dokumentu. TF-IDF (engl. *Term Frequency – Inverse Document Frequency*) nadograđuje ovaj pristup tako što umanjuje važnost reči koje se često pojavljuju u svim dokumentima (poput veznika i predloga), a naglašava reči koje su specifične za određeni dokument.

Oba pristupa imaju zajednički fundamentalni nedostatak: proizvode retke vektore visoke dimenzionalnosti u kojima nema nikakve semantičke informacije. Reči "kamera" i "fotoaparat" tretiraju se kao potpuno različite dimenzije, iako označavaju isti pojam. Posledično, fraze koje su bliske po značenju mogu biti geometrijski daleko, što ih čini nepouzdanim za klasterizaciju zasnovanu na semantičkoj sličnosti. Iz tog razloga nisu od interesa u ovom radu.

5.1.2. *Word Embeddings*

Word2Vec i *GloVe* predstavljaju prekretnicu u reprezentaciji teksta. Umesto retkih vektora, ovi modeli generišu guste vektore male dimenzionalnosti u kojima su reči sličnog značenja geometrijski bliske. Ovo se postiže treniranjem plitkih neuronskih mreža na velikim tekstualnim korpusima, pri čemu model uči da reči koje se pojavljuju u sličnim kontekstima imaju slične vektorske reprezentacije. Najpoznatiji primer je analogija "kralj − muškarac + žena ≈ kraljica", koja ilustruje da vektorski prostor kodira semantičke odnose između reči, što je prikazano na slici 5.2.



Slika 5.2. Ilustracija semantičkih odnosa u Word2Vec vektorskom prostoru [11]

Međutim, *word embeddings* imaju važno ograničenje: svaka reč dobija jedan fiksni vektor bez obzira na kontekst u kome se pojavljuje. Takođe, vektori se generišu na nivou pojedinačnih reči. Za potrebe ovog rada, gde su ulazne jedinice kratke fraze poput "trajanje

baterije" ili "kvalitet zvuka", neophodno je imati reprezentaciju na nivou fraze ili rečenice – što *word embeddings* ne pružaju na prirodan način [10].

5.1.3. *Sentence Transformers*

BERT (engl. *Bidirectional Encoder Representations from Transformers*) uveo je kontekstualne vektorske reprezentacije – za razliku od *Word2Vec*-a, isti token dobija različit vektor u zavisnosti od rečenice u kojoj se nalazi, što omogućava znatno bogatiju reprezentaciju značenja. Međutim, originalni BERT nije bio prikladan za direktno generisanje vektora na nivou rečenice: usrednjavanje vektora koji reprezentuju pojedinačne tokene davalo je loše rezultate za zadatke semantičke sličnosti jer ne uzima u obzir redosled reči.

Sentence-BERT (SBERT), rešava ovaj problem modifikacijom BERT arhitekture kroz sijamsku (engl. *siamese*) mrežnu strukturu. Dve rečenice prolaze kroz isti model nezavisno jedna od druge i svaka dobija sopstveni gust vektor. Model se trenira na skupovima rečenica sa poznatim odnosima sličnosti – parovi rečenica koji su semantički bliski guraju se jedni prema drugima u vektorskom prostoru, dok se semantički različiti parovi razdvajaju. Ovakav pristup donosi važnu praktičnu prednost – vektorska reprezentacija svake fraze može se unapred izračunati i sačuvati, a poređenje između bilo koje dve fraze svodi se na jednostavno izračunavanje kosinusne sličnosti između njihovih vektora, što je računski trivijalno [12].

Za razliku od BoW modela koji ignoriše redosled i sintaksu, SBERT uzima u obzir celokupan kontekst fraze, što rezultira znatno preciznijim semantičkim reprezentacijama. U kontekstu ovog rada, to znači da fraze poput "baterija" i "trajanje baterije" dobijaju geometrijski bliske vektore, dok su "baterija" i "ekran" daleke – što je upravo osnova za smislenu klasterizaciju aspektnih fraza.

5.1.4. Poređenje tehnika reprezentacije teksta

Tabela 1 sumira opisane metode reprezentacije teksta i prikazuje ključne razlike među njima, s ciljem da obrazloži zašto je SBERT odabran kao osnova za generisanje vektorskih reprezentacija teksta u ovom radu.

Tabela 1. Poređenje opisanih metoda reprezentacije teksta

Metoda	**Tip vektora**	**Semantička sličnost**	**Osnova reprezentacije**
BoW	redak	ne	frekvencija reči
TF-IDF	redak	delimično	težinska frekvencija reči
Word2Vec* / *GloVe	gust	da	značenje reči
SBERT	gust	da	značenje fraze/rečenice

5.2. Algoritmi klasterizacije

Kao što je ranije spomenuto, klasterizacija je tehnika nenadgledanog mašinskog učenja koja grupiše podatke u klastere tako da su elementi unutar istog klastera međusobno sličniji nego elementi iz različitih klastera. U kontekstu klasterizacije teksta, ulazni podaci su vektorske reprezentacije, a cilj je pronaći grupe koje odgovaraju smislenim semantičkim kategorijama. Postoji više algoritama koji ovaj problem rešavaju na različite načine, a izbor algoritma značajno utiče na kvalitet i interpretabilnost rezultata [10].

K-Means je jedan od najpoznatijih i najčešće korišćenih algoritama klasterizacije. Funkcioniše iterativno: nasumično postavlja *k* centroida, dodeljuje svaku tačku najbližem centroidu, zatim pomera centroide u središte svog klastera i ponavlja postupak do konvergencije. Glavna prednost *K-Means* algoritma je jednostavnost i efikasnost – dobro se skalira na velike skupove podataka. Ključni nedostatak je što broj klastera *k* mora biti poznat unapred, što u praksi nije uvek slučaj. Pored toga, algoritam je osetljiv na inicijalizaciju centroida i pretpostavlja da su klasteri sfernog oblika, što može biti problematično pri radu sa tekstualnim podacima.

DBSCAN (engl. *Density-Based Spatial Clustering of Applications with Noise*) pristupa klasterizaciji na potpuno drugačiji način – umesto centroida, koristi gustinu podataka. Algoritam definiše klastere kao guste regione u prostoru odvojene oblastima male gustine, pri čemu tačke koje ne pripadaju nijednom gustom regionu tretira kao šum i isključuje ih iz klasterizacije. Prednost ovog pristupa je što broj klastera nije potrebno zadati unapred i što algoritam prirodno obrađuje šum. Međutim, DBSCAN je osetljiv na izbor dva ključna parametra – poluprečnika i minimalnog broja tačaka u krugu – čije podešavanje može biti netrivijalno, posebno u visokodimenzionalnim prostorima kao što su vektorske reprezentacije teksta.

GMM (engl. *Gaussian Mixture Models*) modeluju podatke kao kombinaciju više Gausovih raspodela, pri čemu svaki klaster odgovara jednoj raspodeli. Za razliku od *K-Means* algoritma koji svaku tačku striktno dodeljuje jednom klasteru, GMM dodeljuje svim tačkama verovatnoće pripadnosti svakom klasteru – što ga čini fleksibilnijim za podatke koji se prirodno preklapaju. Međutim, GMM takođe zahteva unapred poznat broj klastera i računski je zahtevniji od *K-Means* algoritma.

Agglomerative Clustering je hijerarhijski algoritam koji počinje sa svakim elementom kao zasebnim klasterom i postepeno spaja najsličnije klastere sve dok se ne dostigne željeni broj klastera ili zadati kriterijum zaustavljanja. Ključna prednost je što ne zahteva da broj klastera bude poznat unapred – hijerarhija spajanja može se vizualizovati kroz dendrogram koji omogućava naknadni odabir optimalnog broja klastera. Detaljna analiza ovog algoritma, uključujući strategije spajanja i evaluaciju, data je u narednoj sekciji.

Tabela 2 prikazuje poređenje opisanih algoritama po kriterijumima relevantnim za zadatak klasterizacije aspektnih fraza:

Tabela 2. Poređenje algoritama za klasterizaciju

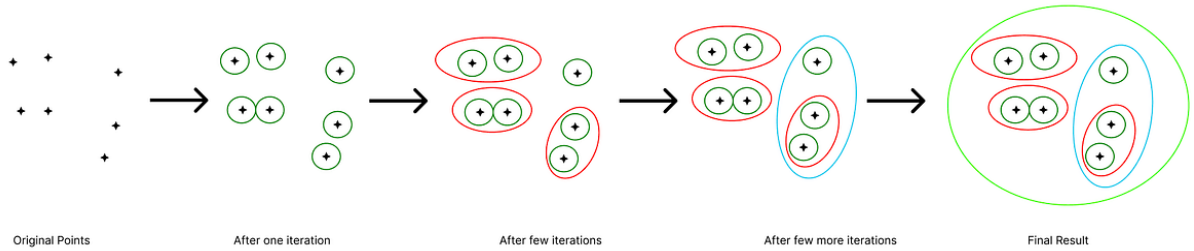
Algoritam	**Zahteva broj klastera unapred**	**Obrada šuma**	**Skalabilnost**	**Interpretabilnost**
K-Means	da	ne	visoka	srednja
DBSCAN	ne	da	srednja	srednja
GMM	da	ne	srednja	srednja
Agglomerative	ne	delimično	niža	visoka (dendrogram)

Iz tabele 2 se vidi da je *Agglomerative Clustering* algoritam koji ne zahteva unapred poznat broj klastera i istovremeno nudi visoku interpretabilnost kroz vizualizaciju hijerarhije spajanja, kao i delimičnu obradu šuma – što su ključni kriterijumi za zadatak klasterizacije aspektnih fraza u ovom radu, gde optimalan broj kategorija nije poznat unapred.

5.3. *Agglomerative Clustering*

Agglomerative Clustering je hijerarhijski algoritam klasterizacije koji funkcioniše po *bottom-up* principu. Za razliku od particijskih algoritama poput *K-Means*-a koji dele podatke na unapred zadati broj grupa, *Agglomerative Clustering* radi obrnuto: svaki element inicijalno čini sopstveni klaster, a algoritam iterativno spaja najsličnije klastere sve dok se ne ispuni zadati kriterijum zaustavljanja. Rezultat nije samo finalna podela podataka, već čitava hijerarhija spajanja koja pruža uvid u strukturu podataka na različitim nivoima granularnosti.

Na početku algoritma, svaki od n elemenata predstavlja zaseban klaster. Algoritam zatim računa rastojanja između svih parova klastera i formira matricu rastojanja dimenzije $n \times n$. U svakoj iteraciji pronalaze se dva klastera sa najmanjim međusobnim rastojanjem i spajaju u jedan novi klaster. Matrica rastojanja se ažurira tako da odražava rastojanja između novonastalog klastera i svih ostalih, a proces se ponavlja sve dok ne ostane jedan klaster koji obuhvata sve elemente, ili dok se ne dostigne željeni broj klastera (ilustrovano na slici 5.3) [14]. Način na koji se računa rastojanje između klastera određuje se strategijom spajanja koja je detaljno opisana u nastavku.

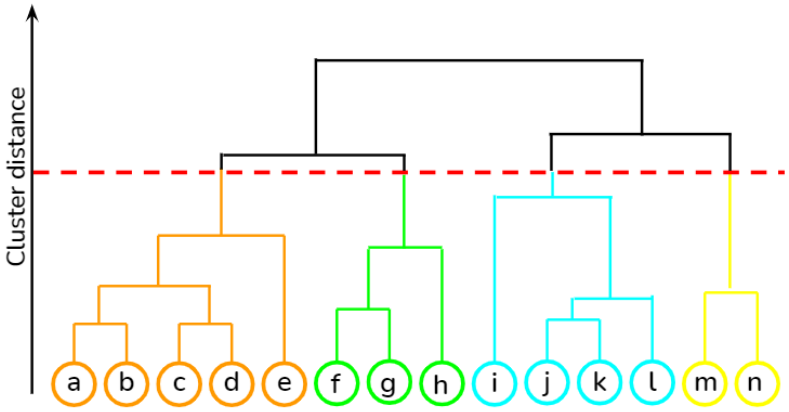


Slika 5.3. Proces rada Agglomerative Clustering algoritma [15]

5.3.1. Dendrogrami

Hijerarhija spajanja koju algoritam gradi tokom izvršavanja može se vizualizovati kroz dendrogram – dijagram u obliku stabla koji prikazuje redosled i rastojanja spajanja klastera. Na X osi nalaze se pojedinačni elementi, a Y osa predstavlja rastojanje na kome je određeno spajanje izvršeno. Svako grananje dendrograma odgovara jednom koraku spajanja: što je spajanje više na Y osi, to su dva klastera bila međusobno udaljenija u trenutku spajanja.

Dendrogram je posebno vredan jer omogućava naknadni odabir optimalnog broja klastera bez ponovnog pokretanja algoritma. Optimalan broj klastera određuje se horizontalnim presekom dendrograma – linija se postavlja na mestu najvećeg skoka na Y osi, što odgovara spajanju koje je premostilo najveće rastojanje i time označava prirodnu granicu između grupa. Broj vertikalnih linija koje horizontalni rez preseca odgovara broju klastera. Dendrogram sa horizontalnim presekom je prikazan na slici 5.4.



Slika 5.4. Određivanje optimalnog broja klastera horizontalnim presekom dendrograma [16]

5.3.2. Strategije spajanja

Strategija spajanja (engl. *linkage*) definiše kako se meri rastojanje između dva klastera koji se razmatraju za spajanje. Izbor strategije značajno utiče na oblik i kvalitet dobijenih klastera.

Single linkage definiše rastojanje između dva klastera kao rastojanje između njihova dva najbliža elementa. Ova strategija dobro funkcioniše za klastere nepravilnog oblika i osetljiva je na lokalne veze između podataka. Međutim, sklona je pojavi tzv. efekta lanca (engl. *chaining effect*) – situacije u kojoj se klasteri postepeno produžuju spajanjem elemenata koji su blizu ivice grupe, što može rezultirati dugačkim, razvučenim klasterima bez jasne strukture.

Complete linkage definiše rastojanje kao rastojanje između dva najudaljenija elementa iz različitih klastera. Ovaj pristup teži ka formiranju kompaktnih, ujednačenih klastera i otporniji je na efekat lanca. Mana je što može biti osetljiv na prisustvo *outlier*-a – jedan udaljeni element može značajno povećati rastojanje između klastera i time odložiti njihovo spajanje.

Average linkage koristi prosečno rastojanje između svih parova elemenata iz dva klastera. Predstavlja kompromis između *single* i *complete linkage* – manje je osetljiv na *outlier*-e od *complete linkage* i manje sklon efektu lanca od *single linkage*. Daje uravnotežene rezultate u većini situacija i često se preporučuje kao podrazumevano rešenje kada nema jakih pretpostavki o obliku klastera.

Ward linkage za razliku od prethodnih strategija ne meri direktno rastojanje između klastera, već minimizuje ukupnu varijansu unutar klastera nakon spajanja. U svakom koraku spajaju se ona dva klastera čije bi spajanje dovelo do najmanjeg porasta ukupne varijanse unutar klastera. Ova strategija teži ka formiranju kompaktnih klastera sličnih veličina i generalno daje dobre rezultate za globalno raspoređene podatke, ali pretpostavlja euklidsko rastojanje što je može činiti manje prikladnom u određenim vektorskim prostorima.

5.3.3. Prednosti i mane

Agglomerative Clustering nudi nekoliko važnih prednosti u kontekstu klasterizacije tekstualnih podataka. Broj klastera ne mora biti poznat unapred – dendrogram omogućava da se taj odabir donese naknadno, na osnovu vizualizacije hijerarhije. Algoritam je deterministički, što znači da pri istim ulaznim podacima uvek daje iste rezultate, za razliku od *K-Means*-a koji zavisi od nasumične inicijalizacije. Pored toga, hijerarhijska struktura rezultata pruža bogatiji uvid u organizaciju podataka od ravnih (engl. *flat*) podela koje nude particioni algoritmi.

Glavna mana je računska složenost, što ga čini nepraktičnim za vrlo velike skupove podataka. Kvalitet rezultata takođe zavisi od izbora strategije spajanja, što zahteva određeno poznavanje strukture podataka ili eksperimentisanje. Konačno, jednom izvršeno spajanje ne može se poništiti – lokalno loša odluka u ranoj fazi algoritma može negativno uticati na finalnu klasterizaciju.

5.4. Metrike kvaliteta klasterizacije

Evaluacija kvaliteta klasterizacije je teža od evaluacije nadgledanih metoda mašinskog učenja jer ne postoje tačne oznake sa kojima bi se rezultati mogli porediti. Umesto toga, koriste se interne metrike koje procenjuju kvalitet klasterizacije isključivo na osnovu strukture samih podataka, mereći koliko su klasteri interno kohezivni i međusobno dobro razdvojeni [17]. U nastavku su opisane dve metrike koje su prikladne za upotrebu sa *Agglomerative Clustering* algoritmom, pri čemu se jedna od njih koristi u ovom radu za odabir optimalnog broja klastera.

Silhouette score meri koliko dobro svaki element pripada svom klasteru u poređenju sa susednim klasterima. Za svaki element računaju se dve vrednosti: prosečno rastojanje do ostalih elemenata unutar istog klastera (a), i prosečno rastojanje do elemenata najbližeg susednog klastera (b). *Silhouette score* za taj element definiše se kao:

$$s = \frac{b - a}{\max(a, b)}$$

Vrednost se kreće u opsegu od -1 do 1. Vrednosti bliske 1 ukazuju da je element dobro smešten unutar svog klastera i daleko od susednih klastera. Vrednosti bliske 0 sugerišu da se element nalazi na granici između dva klastera. Negativne vrednosti ukazuju da je element možda pogrešno raspoređen – bliži je elementima susednog klastera nego sopstvenog. Ukupni *Silhouette score* klasterizacije dobija se usrednjavanjem individualnih vrednosti svih elemenata. Za odabir optimalnog broja klastera, algoritam se pokreće za različite vrednosti broja klastera, a *Silhouette score* se beleži za svaki pokušaj. Optimalan broj klastera odgovara vrednosti pri kojoj je *score* najveći. Ova strategija se koristi u praktičnom delu ovog rada.

Davies-Bouldin indeks meri prosečnu sličnost svakog klastera sa njemu najsličnijim susednim klasterom, gde se sličnost definiše kao odnos između unutarklasterskog rasipanja i međuklasterskog rastojanja. Za svaki klaster pronalazi se susedni klaster koji je najteže razlikovati od njega, a indeks predstavlja prosek tih najgorih slučajeva kroz sve klastere.

Za razliku od *Silhouette score*-a, *Davies-Bouldin* indeks nema fiksiran gornji opseg – vrednosti su nenegativne, a niža vrednost ukazuje na bolju klasterizaciju. Vrednost 0 bila bi teorijski savršena klasterizacija sa potpuno razdvojenim i kompaktnim klasterima. Ova metrika je posebno osetljiva na slučajeve kada su klasteri međusobno bliski i interno rasuti, što je čini korisnim komplementom *Silhouette score*-u.

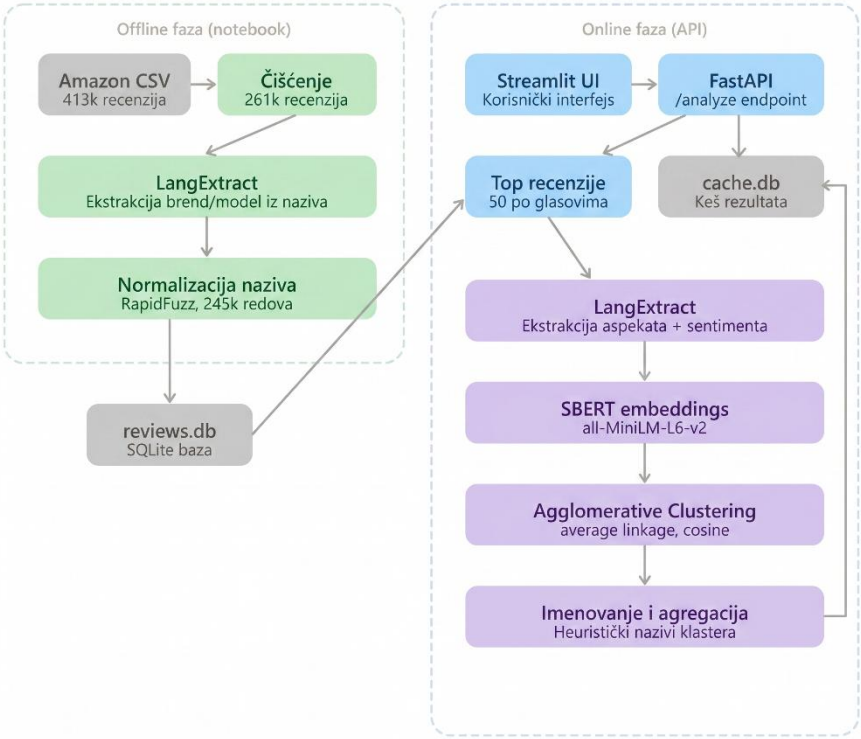
U praksi, ove dve metrike se koriste zajedno jer mere različite aspekte kvaliteta klasterizacije i mogu dati potpuniju sliku. *Silhouette score* daje globalnu ocenu koliko su klasteri dobro razdvojeni, dok *Davies-Bouldin* indeks ističe problematične parove klastera koji su teško razdvojivi. Optimalan broj klastera bira se kao vrednost pri kojoj je *Silhouette score* visok, a *Davies-Bouldin* indeks nizak.

6. PRAKTIČNI DEO

Teorijska osnova izložena u prethodnim poglavljima – ekstrakcija informacija, analiza sentimenta na nivou aspekta, semantičke reprezentacije teksta i klasterizacija – u ovom poglavlju se objedinjuje u konkretan sistem za automatizovanu analizu korisničkih recenzija mobilnih telefona. Sistem je implementiran kao *web* aplikacija koja korisniku omogućava da odabere proizvođača i model uređaja i dobije pregled najčešće pominjanih karakteristika tog uređaja, zajedno sa informacijom o tome koliko su korisnici prema svakoj karakteristici pozitivno ili negativno reagovali. U nastavku su opisane sve faze razvoja sistema – od pripreme podataka i ekstrakcije aspekata, preko klasterizacije i agregacije rezultata, do konačnog prikaza u korisničkom interfejsu.

6.1. Opis sistema i arhitektura

Sistem je organizovan tako da razdvaja računski intenzivne operacije koje se izvršavaju jednom od operacija koje se pokreću na zahtev korisnika. Na slici 6.1. je prikazana arhitektura sistema.



Slika 6.1. Arhitektura sistema

***Offline* faza** izvršava se jednom, pre pokretanja aplikacije, i njena svrha je priprema skupa podataka. U ovoj fazi se učitava sirovi CSV fajl sa *Amazon* recenzijama, vrši se čišćenje

i filtriranje podataka, a zatim se primenom *LangExtract* biblioteke iz naziva proizvoda ekstrahuju nazivi brendova i modela. Ova upotreba *LangExtract*-a je relativno jednostavna – biblioteka se koristi za strukturirano parsiranje kratkih tekstualnih polja – dok se njena složenija i centralnija primena, ekstrakcija aspekata i sentimenta iz dugačkih recenzija, odvija u *online* fazi. Nakon normalizacije naziva, podaci se čuvaju u *SQLite* bazi podataka koja služi kao osnova za *online* fazu. *Offline* faza implementirana je kao *Jupyter notebook* i nije deo same *web* aplikacije.

Online **faza** pokreće se na zahtev korisnika kroz *web* interfejs. Korisnik bira brend i model uređaja, nakon čega sistem preuzima najrelevantnije recenzije iz baze, ekstrahuje aspekte i sentiment primenom *LangExtract* biblioteke, grupiše semantički slične aspekte klasterizacijom i vraća agregirane rezultate. Ceo ovaj proces odvija se kroz *FastAPI backend* koji izlaže REST API, dok *Streamlit* aplikacija služi kao korisnički interfejs koji poziva taj API.

Da bi se izbeglo ponavljanje skupih operacija – pre svega paralelnih *LangExtract* poziva koji su vremenski i finansijski najzahtevniji – sistem koristi mehanizam keširanja zasnovan na posebnoj *SQLite* bazi. Kada korisnik zatraži analizu određenog modela, sistem najpre proverava da li postoji važeći keširani rezultat. Keš se smatra važećim ako nije stariji od tri dana, a može se prinudno osvežiti parametrom *force_refresh*. Ako validni keš postoji, rezultat se vraća odmah bez pokretanja ekstrakcije. U suprotnom, sistem prolazi kroz ceo *pipeline* i na kraju upisuje rezultat u keš za buduće zahteve.

Komunikacija između komponenti sistema odvija se na sledeći način: *Streamlit frontend* poziva tri API *endpoint*-a – */brands* za listu dostupnih brendova, */models* za listu modela odabranog brenda, i */analyze* koji pokreće ili vraća keširanu analizu. *Backend* pri pokretanju inicijalizuje keš bazu i unapred učitava SBERT model u memoriju kako bi se izbeglo kašnjenje pri prvom korisničkom zahtevu i kako se model ne bi učitavao pri svakom korisničkom zahtevu.

6.2. Skup podataka

Za potrebe ovog rada korišćen je javno dostupan skup podataka *Amazon Unlocked Mobile Phones* dostupan na platformi *Kaggle* [18]. Skup sadrži korisničke recenzije mobilnih telefona prikupljene sa *Amazon* platforme i predstavlja realan, nestrukturirani tekstualni sadržaj koji dobro odražava izazove sa kojima se sistemi za analizu sentimenta susreću u praksi.

Sirovi skup podataka sadrži 413.840 redova i šest kolona: naziv proizvoda (*Product Name*), naziv brenda (*Brand Name*), cena (*Price*), ocena (*Rating*), tekst recenzije (*Reviews*) i broj glasova koji označava koliko je korisnika smatralo recenziju korisnom (*Review Votes*). Kolona *Review Votes* ima poseban značaj za ovaj sistem – koristi se za određivanje kvaliteta i relevantnosti recenzije, pa se pri analizi određenog modela telefona preuzima prvih 50 recenzija sortiranih po ovoj vrednosti, čime se prioritizuje sadržaj koji je zajednica korisnika ocenila kao najkorisniji.

Priprema skupa podataka odvijala se kroz nekoliko faza čišćenja i filtriranja. U prvom koraku uklonjeni su redovi sa nedostajućim tekstom recenzije ili nazivom brenda. Zatim su

filtrirane recenzije koje su prekratke – manje od 5 reči – jer ne sadrže dovoljno informacija za smislenu ekstrakciju aspekata, kao i recenzije koje imaju formu pitanja, prepoznate po tome što počinju upitnom rečju ili se završavaju upitnikom. Ovaj korak je važan jer pitanja poput "Da li ovaj telefon ima dobru kameru?" ne izražavaju nikakav sentiment već traže informaciju. Nakon čišćenja ostalo je 261.723 recenzija, što znači da je uklonjeno 36,8% originalnih redova.

Poseban izazov predstavljala je kolona *Product Name*, koja nije sadržala standardizovane nazive modela već sirove, neuredne tekstualne opise kakvi se nalaze na *Amazon* platformi – poput "*Samsung Galaxy S7 Edge G935F 32GB Factory Unlocked Phone – International Version (Black)*". Iz ovakvih opisa bilo je potrebno automatski izvući naziv brenda i modela kako bi se recenzije mogle grupisati po uređajima. Za ovaj zadatak korišćena je *LangExtract* biblioteka koja je obrađivala 3.629 jedinstvenih naziva proizvoda u paralelnim serijama, uz čuvanje međurezultata u *checkpoint* fajlu radi mogućnosti nastavka u slučaju prekida. Od ukupnog broja, 13 naziva nije uspešno obrađeno, a za 310 nije bilo moguće utvrditi naziv modela. Nakon uklanjanja redova bez naziva modela, finalni skup podataka je sadržao 245.560 recenzija.

Nakon toga, postojao je problem što su isti brendovi u skupu podataka bili zapisani na različite načine, na primer: "Samsung", "samsung", "SAMSUNG" i "Samssung". Normalizacija naziva brendova sprovedena je u dva koraka: najpre grupisanjem varijanti koje se razlikuju samo po veličini slova, a zatim primenom algoritma za *fuzzy* podudaranje (*RapidFuzz*, prag sličnosti 85) kojim su popravljene tipografske greške i neznatne varijacije u pisanju istog naziva. Nazivi modela normalizovani su mapiranjem svih varijanti pisanja na najčešće zastupljenu formu. Ovaj korak bio je neophodan jer nekonzistentnost u pisanju naziva može dovesti do toga da se recenzije istog uređaja raspršе u više zasebnih grupa i time naruše kvalitet analize. Sličan postupak je primenjen i za normalizaciju modela telefona. Konačan skup podataka je sačuvan u *SQLite* bazi podataka, radi brze i efikasne pretrage i pribavljanja odgovarajućih recenzija pri analizi.

6.3. Struktura baza podataka

Sistem koristi dve odvojene *SQLite* baze podataka koje imaju jasno odvojene uloge: jedna služi kao izvor recenzija, a druga kao keš sloj koji ubrzava ponovljene upite.

Baza ***reviews.db*** proizvod je *offline* faze i jednom kreirana ne menja se tokom rada aplikacije. Sadrži tabelu *reviews* sa sledećim kolonama: *id*, *Product Name*, *Brand Name*, *Price, Rating, Reviews, Review Votes, review_word_count* i *model*. Kolona *model* rezultat je *LangExtract* ekstrakcije opisane u prethodnoj sekciji i predstavlja normalizovani naziv modela koji se koristi za grupisanje recenzija.

Za efikasno pretraživanje po brendu i modelu kreiran je kompozitni indeks *idx_brand_model* na kolonama (*Brand Name, model*). Ovaj indeks je ključan za performanse *online* faze. Pre prosleđivanja *LangExtract* biblioteci, tekst svake recenzije se skraćuje na maksimalno 500 reči jer se smatra da se u prvih 500 reči nalazi većina bitnih utisaka, a i kako bi se kontrolisali troškovi i vreme obrade.

Baza ***cache.db*** kreira se i ažurira tokom rada aplikacije i čuva rezultate analize kako bi se izbeglo ponavljanje skupih *LangExtract* poziva za iste upite. Organizovana je u dve međusobno povezane tabele.

Tabela *phones* čuva metapodatke o svakom analiziranom uređaju: naziv brenda i modela, parametar *top_n* koji označava koliko je maksimalno recenzija analizirano, broj recenzija koji je zaista analiziran, kao i vremensku oznaku poslednjeg keširanja *last_cached_at*. Kombinacija kolona (*brand, model, top_n*) mora biti jedinstvena – isti upit sa istim parametrima uvek vraća isti keširani rezultat sve dok nije istekao rok važenja od tri dana.

Tabela *clusters* čuva agregirane rezultate klasterizacije vezane za određeni unos u tabeli *phones*. Za svaki klaster čuva se reprezentativni naziv (*representative_name*), broj pozitivnih, negativnih i neutralnih pominjanja, ukupan broj pominjanja (*total_mentions*) i lista originalnih fraza koje su dodeljene tom klasteru (*original_mentions*) sačuvana kao JSON niz tekstova. Ovakva struktura omogućava brzu rekonstrukciju API odgovora bez ponovnog pokretanja *pipeline*-a.

6.4. Ekstrakcija karakteristika

Nakon što su recenzije pripremljene i smeštene u bazu, sledeći korak u *pipeline*-u je ekstrakcija aspekata i sentimenta iz teksta svake recenzije. Ova faza je računski najzahtevniji deo sistema i direktno se oslanja na *LangExtract* biblioteku opisanu u teorijskom delu rada.

6.4.1. Definisanje zadatka ekstrakcije

Zadatak ekstrakcije definisan je kroz upit i četiri primera koji zajedno opisuju šta sistem treba da izvuče iz teksta. Upit govori modelu da ekstrahuje karakteristike telefona kao koncizne fraze od jedne do tri reči, da se fokusira isključivo na karakteristike samog uređaja, da spaja sinonime i izbegava kopiranje dugačkih rečenica, i da sentiment klasifikuje kao *positive*, *negative* ili *neutral* na osnovu konteksta. Svaka ekstrakcija sastoji se od klase *aspect*, tačnog teksta iz izvora i atributa sa vrednošću sentimenta.

Primeri prosleđeni biblioteci pokrivaju različite kombinacije sentimenta kako bi model naučio očekivani format: baterija sa pozitivnim sentimentom i kamera sa negativnim, ekran sa pozitivnim i brzina punjenja sa neutralnim, kvalitet zvuka i cena oba sa negativnim, i kvalitet izrade sa pozitivnim sentimentom. Ovakav skup primera osigurava da model razume kako da tretira mešovite recenzije u kojima isti tekst sadrži i pohvale i kritike.

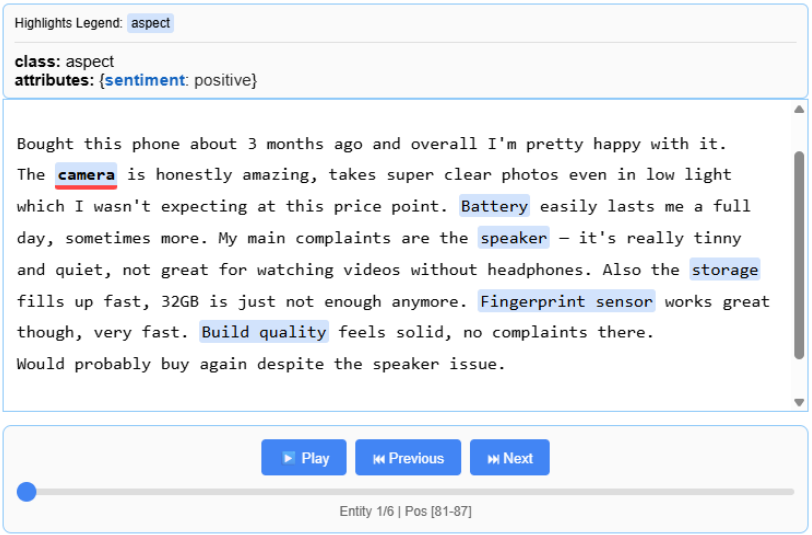
6.4.2. Paralelna obrada recenzija

Za svaki upit korisnika koji nije keširan sistem preuzima do 50 recenzija sortiranih po broju korisnih glasova opadajuće. Ekstrakcija se pokreće paralelno nad svim recenzijama korišćenjem *ThreadPoolExecutor* sa brojem radnih niti jednakim broju recenzija. Svaka nit izvršava jedan *LangExtract* poziv nezavisno, a rezultati se prikupljaju po završetku svih poziva. Greške na nivou pojedinačne recenzije ne zaustavljaju ceo proces – neuspešni pozivi vraćaju praznu listu i *pipeline* nastavlja sa ostalim recenzijama. Na ovaj način sistem ostaje robustan čak i kada pojedini *LangExtract* pozivi ne uspeju zbog mrežnih ili API grešaka.

Nakon završetka paralelne ekstrakcije, rezultati svih recenzija se spajaju u jedinstvenu listu parova (*aspect, sentiment*) koja se prosleđuje sledećoj fazi – generisanju vektorskih reprezentacija i klasterizaciji.

6.4.3. Primer ekstrakcije

Slika 6.2. prikazuje HTML vizualizaciju koju *LangExtract* generiše za jednu recenziju, gde su izvučeni aspekti označeni plavom bojom direktno u izvornom tekstu. Za svaki aspekt je moguće videti da li je pomenut u pozitivnom ili negativnom kontekstu. Vizualizacija omogućava laku proveru kvaliteta ekstrakcije – korisnik može direktno videti koje fraze iz originalne recenzije su prepoznate kao aspekti i da li im je sentiment ispravno dodeljen. Pored vizuelnog prikaza, svaki označeni aspekt može se pregledati i u strukturiranom obliku koji prikazuje klasu ekstrakcije, tačan tekst i pridruženi sentiment atribut.



Slika 6.2. LangExtract HTML vizualizacija ekstrakcije aspekata iz recenzije mobilnog telefona

6.5. Vektorske reprezentacije i klasterizacija

Nakon što se paralelna ekstrakcija završi, sistem raspolaže listom sirovih aspektnih fraza sa pridruženim sentiment vrednostima. Ove fraze su nekonzistentne po formi – različiti korisnici isti aspekt opisuju različitim rečima – pa ih pre agregacije treba grupisati u smislene semantičke kategorije. Ova faza odvija se u dva koraka: generisanje vektorskih reprezentacija fraza i njihova klasterizacija.

6.5.1. Generisanje vektorskih reprezentacija fraza

Za generisanje vektorskih reprezentacija koristi se pretreniran model *all-MiniLM-L6-v2* iz *sentence-transformers* biblioteke. Ovaj model je odabran zbog povoljnog odnosa između brzine i kvaliteta – generiše vektore dimenzije 384 i optimizovan je za zadatke semantičke sličnosti na kratkim tekstovima, što ga čini pogodnim za aspektne fraze koje su tipično dugačke jednu do tri reči.

Pored toga, implementirana je optimizacija generisanja vektorskih reprezentacija: pre prosleđivanja modelu, lista fraza se svodi na skup jedinstvenih vrednosti, vektori se generišu samo za te jedinstvene fraze, a zatim se mapiraju nazad na originalnu listu. S obzirom da iste fraze poput "*battery life*" ili "*camera*" mogu da se pojave u recenzijama desetinama puta, ova optimizacija značajno smanjuje broj potrebnih poziva modelu bez ikakvog uticaja na rezultat.

6.5.2. Konfiguracija klasterizacije

Klasterizacija se vrši primenom *Agglomerative Clustering* algoritma iz *scikit-learn* biblioteke sa konfiguracijom prikazanom na slici 6.3.

```
AgglomerativeClustering(
    n_clusters=None,
    distance_threshold=optimal_threshold,
    metric='cosine',
    linkage='average'
)
```

Slika 6.3. Konfiguracija AgglomerativeClustering algoritma u projektu

Parametar *n_clusters=None* u kombinaciji sa *distance_threshold* znači da algoritam sam određuje broj klastera na osnovu podataka – spajanje se nastavlja sve dok kosinusna sličnost između klastera ne padne ispod prihvatljivog nivoa, odnosno dok kosinusno rastojanje ne premaši zadati prag. Kada se taj prag dostigne, spajanje se zaustavlja i trenutna podela se

uzima kao finalna klasterizacija. Ovo je ključna odluka u dizajnu sistema: broj kategorija aspekata nije poznat unapred i razlikuje se od uređaja do uređaja, pa bi ručno zadavanje broja klastera bilo nepraktično i nepouzdano.

Umesto korišćenja fiksne vrednosti praga rastojanja, sistem automatski određuje optimalni *threshold* u opsegu od 0.5 do 0.6 sa korakom 0.05. Za svaku kandidat vrednost pokreće se klasterizacija i izračunava *Silhouette score* koji meri koliko su klasteri interno kohezivni i međusobno dobro razdvojeni. Bira se *threshold* koji daje najveći *Silhouette score*, čime se broj klastera prilagođava konkretnom skupu aspektnih fraza za svaki upit. Uski opseg pretrage od 0.5 do 0.6 odabran je eksperimentalno kao zona u kojoj klasterizacija daje smislene rezultate – vrednosti ispod 0.5 tipično proizvode previše sitnih klastera, dok vrednosti iznad 0.6 spajaju konceptualno različite aspekte. Ako ni jedan *threshold* u opsegu ne daje validnu klasterizaciju – što se može desiti kada je broj jedinstvenih aspekata premali – sistem koristi podrazumevanu vrednost od 0.5.

Kosinusna metrika rastojanja odabrana je jer meri ugaonu sličnost između vektora, što je prikladnije od euklidskog rastojanja u visokodimenzionalnim prostorima kakve su semantičke vektorske reprezentacije. Strategija spajanja *average linkage* računa prosečno rastojanje između svih parova elemenata iz dva klastera, što daje uravnotežene rezultate i manje je osetljiva na prisustvo *outlier*–a od *complete* ili *single linkage* strategija.

Nakon klasterizacije, klasteri koji sadrže samo jedno pominjanje se odbacuju. Ovakve izolovane fraze najčešće su rezultat specifičnih komentara koji se ne ponavljaju kroz recenzije i ne nose statistički smislenu informaciju o opštoj percepciji uređaja.

6.6. Imenovanje klastera i agregacija sentimenta

Nakon klasterizacije, svaki klaster sadrži grupu semantički sličnih aspektnih fraza koje zajedno predstavljaju jednu kategoriju karakteristika proizvoda. Preostaju dva zadatka: dodeliti klasteru reprezentativni naziv koji će biti prikazan korisniku, i agregirati sentiment svih fraza unutar klastera kako bi se dobio konačni uvid u to kako korisnici doživljavaju tu karakteristiku.

6.6.1. Imenovanje klastera

Imenovanje klastera implementirano je heurističkim pristupom koji se odvija kroz tri koraka. U prvom koraku sistem proverava da li postoji dominantna reč – reč koja nije stop reč i koja zauzima najmanje 50% reči unutar klastera. Ako takva reč postoji, koristi se kao reprezentativni naziv. Na primer, ako klaster sadrži fraze “*battery life*”, “*battery drain*” i “*battery*”, reč "*battery*" pojavljuje se najčešće i prirodno postaje naziv klastera.

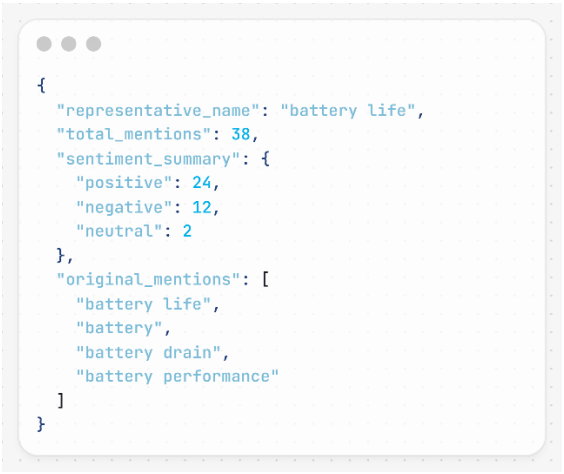
Ako dominantna reč ne postoji, sistem bira najfrekventniju celu frazu unutar klastera kao reprezentativni naziv. Ovaj korak pokriva slučajeve gde su fraze raznovrsnije, ali jedna od njih jasno dominira po učestalosti.

Ako ni taj kriterijum ne daje jednoznačan rezultat, sistem onda koristi kosinusnu sličnost sa centroidom klastera – bira se fraza čiji je vektor najbliži proseku svih vektora unutar klastera, što je u suštini fraza koja najbolje reprezentuje semantički centar grupe.

6.6.2. Agregacija sentimenta

Za svaki klaster sistem broji koliko puta su fraze koje mu pripadaju bile označene kao pozitivne, negativne i neutralne. Ovo brojanje vrši se jednostavnim prebrojavanjem sentiment vrednosti svih pominjanja u klasteru primenom *Counter* strukture. Rezultat je rečnik sa ključevima *positive*, *negative* i *neutral* i odgovarajućim brojevima pominjanja.

Finalni izlaz sistema za svaki klaster sadrži reprezentativni naziv, ukupan broj pominjanja, raspodelu sentimenta i listu originalnih fraza koje su mu dodeljene. Klasteri su sortirani opadajuće po ukupnom broju pominjanja, što znači da su najčešće diskutovane karakteristike prikazane prve. Primer na slici 6.4. ilustruje tipičan izlaz za jedan klaster. Niz ovakvih objekata se prosleđuje *Streamlit* frontendu koji ga vizualizuje kao pregled aspekata sa sentiment raspodelom za odabrani uređaj.



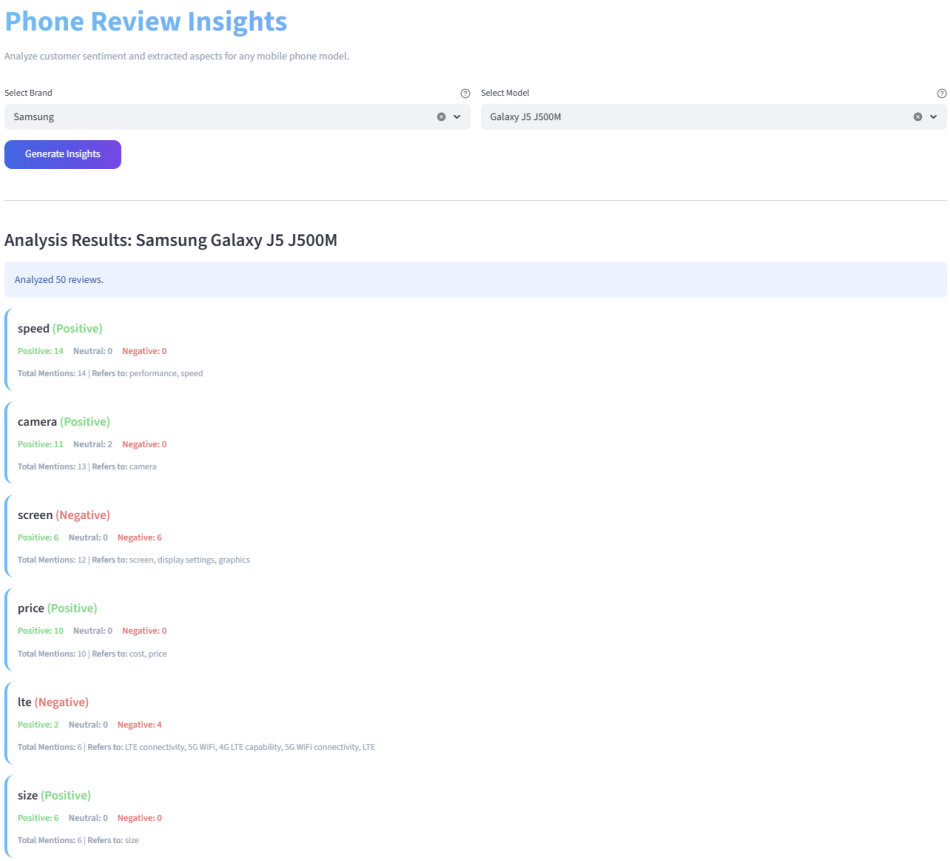
Slika 6.4. Primer tipičnog izlaza za jedan klaster

6.7. Rezultati i evaluacija

Ovaj odeljak prikazuje rezultate rada sistema na konkretnim primerima iz skupa podataka i daje kvalitativnu procenu kvaliteta ekstrakcije i klasterizacije. Budući da sistem ne proizvodi jednoznačan numerički izlaz koji bi se mogao porediti sa referentnim vrednostima – radi se o nenadgledanom pristupu bez tačnih oznaka – evaluacija se zasniva na ručnoj inspekciji rezultata i analizi tipičnih obrazaca koje sistem prepoznaje.

6.7.1. Primer analize konkretnog uređaja

Na slici 6.5. je prikazan korisnički interfejs razvijene aplikacije. Slika prikazuje rezultate analize za odabrani uređaj iz skupa podataka. Može se videti lista klastera sortiranih opadajuće po ukupnom broju pominjanja, pri čemu svaki klaster prikazuje reprezentativni naziv aspekta i raspodelu sentimenta kroz pozitivna, negativna i neutralna pominjanja. Aspekti poput brzine, kamere i ekrana tipično se nalaze na vrhu liste kao najčešće diskutovane karakteristike, dok su aspekti poput kvaliteta zvuka i dizajna ređe pominjani. Na slici je prikazano samo prvih 6 klastera, dok ih je ukupno bilo oko 20.



Slika 6.5. Korisnički interfejs aplikacije

6.7.2. Diskusija kvaliteta klasterizacije

Analizom rezultata može se zaključiti da sistem uspešno grupiše leksički različite ali semantički srodne fraze u iste klastere. Tipičan primer uspešnog grupisanja sa slike 6.5. su fraze "*screen*", "*display settings*" i "*graphics*" koje sistem konzistentno svrstava u jedan klaster, dodeljujući mu naziv "*screen*" kao najadekvatniju frazu.

Grupisanje pokazuje slabosti u slučajevima kada su fraze konceptualno bliske ali kontekstualno različite. Na primer, "*price*" i "*value for money*" ponekad završavaju u istom klasteru iako korisnici kroz ove fraze često izražavaju različite stavove – cena se pominje i u pozitivnom i negativnom kontekstu, dok "*value for money*" gotovo uvek nosi pozitivnu konotaciju. U ovakvim slučajevima agregirani sentiment klastera može biti manje informativan nego što bi bio kada bi ove fraze bile razdvojene.

Heuristički mehanizam imenovanja klastera generalno daje razumljive nazive, ali postoje granični slučajevi. Kada fraze unutar klastera nisu dominantne po jednoj reči niti po jednoj frazi, sistem bira frazu koja je matematički centralna u vektorskom prostoru, što ne mora uvek biti i najprirodniji naziv klastera za korisnički prikaz.

6.7.3. Ograničenja sistema

Nekoliko ograničenja vrednih pažnje proizilazi iz dizajnerskih odluka donesenih tokom razvoja sistema.

Iako je uveden mehanizam automatskog određivanja optimalnog *threshold*-a u opsegu od 0.5 do 0.6, ovaj opseg i dalje je ručno definisan i ne garantuje optimalne rezultate za sve uređaje i sve skupove recenzija. Uređaji sa manjim brojem recenzija ili specifičnijim žargonom mogu zahtevati drugačiji opseg od onog koji daje dobre rezultate na većini uređaja.

Sistem analizira isključivo prvih 50 recenzija sortiranih po broju korisnih glasova. Ovakav odabir uvodi potencijalnu pristrasnost – starije recenzije koje su akumulirale više glasova tokom vremena mogu biti zastupljenije od novijih, što znači da sistem možda ne odražava aktuelno stanje percepcije uređaja, posebno ako je uređaj doživeo softverske nadogradnje koje su promenile korisničko iskustvo.

Kvalitet ekstrakcije direktno zavisi od LLM-a koji se koristi u pozadini. Iako *LangExtract* sa *Gemini* modelom pokazuje dobre rezultate, mogući su propusti u ekstrakciji – aspekti koji se pominju implicitno ili kroz idiomatske izraze mogu biti preskočeni – kao i greške u klasifikaciji sentimenta u slučajevima negacije ili sarkazma, što su izazovi koji su detaljno opisani u teorijskom delu rada.

7. ZAKLJUČAK

Ovaj rad opisuje projektovanje i implementaciju sistema za automatizovanu ekstrakciju i analizu aspekata proizvoda iz korisničkih recenzija. Kombinovanjem *LangExtract* biblioteke za ekstrakciju strukturiranih informacija, SBERT modela za semantičku reprezentaciju teksta i *Agglomerative Clustering* algoritma za grupisanje srodnih aspekata, sistem uspešno prevazilazi ograničenja tradicionalne analize sentimenta na nivou dokumenta i pruža granularni uvid u korisničko iskustvo za konkretne mobilne uređaje. Rezultat je funkcionalna *web* aplikacija koja korisniku omogućava da u nekoliko sekundi dobije pregled najčešće pominjanih karakteristika odabranog uređaja sa raspodelom pozitivnih i negativnih pominjanja po svakom aspektu.

Sistem demonstrira da je moguće izgraditi *pipeline* za analizu sentimenta na nivou aspekata bez ručnog anotiranja podataka i bez domenski specifičnog treniranja modela – *LangExtract* sa odgovarajuće formulisanim upitom i primerima obavlja ekstrakciju, dok SBERT model i klasterizacija rešavaju problem leksičke raznovrsnosti izvučenih fraza. Mehanizam keširanja i paralelna obrada recenzija čine sistem bržim i efikasnijim.

Kao što je naznačeno u prethodnom poglavlju, sistem nosi određena ograničenja. Kvalitet rezultata zavisi od LLM-a koji se koristi za ekstrakciju, heuristički mehanizam imenovanja klastera ne garantuje uvek optimalne nazive, a analiza zasnovana isključivo na top 50 recenzija može unositi pristrasnost.

Prirodni pravci daljeg razvoja uključuju proširenje sistema na druge kategorije proizvoda van mobilnih telefona, uvođenje vremenskog filtriranja recenzija kako bi se pratila evolucija korisničke percepcije tokom vremena, kao i implementaciju *Davies-Bouldin* indeksa kao komplementarne metrike uz *Silhouette score* za robusniju evaluaciju klasterizacije. Na nivou korisničkog interfejsa, vredan pravac razvoja bio bi prikaz reprezentativnih citata iz recenzija za svaki aspekt, čime bi korisnik dobio ne samo agregirani sentiment već i konkretan kontekst iza brojeva.

LITERATURA

[1] "Web Mining," GeeksForGeeks, 6 January 2026. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/web-mining/. [Accessed 7 April 2026].

[2] "What is sentiment analysis?," IBM, [Online]. Available: https://www.ibm.com/think/topics/sentiment-analysis. [Accessed 8 April 2026].

[3] "What is sentiment analysis?," elastic, [Online]. Available: https://www.elastic.co/what-is/sentiment-analysis. [Accessed 8 April 2026].

[4] F. Chiusano, "Quick intro to Aspect-Based Sentiment Analysis," Medium, 19 August 2022. [Online]. Available: https://medium.com/nlplanet/quick-intro-to-aspect-based-sentiment-analysis-c8888a09eda7. [Accessed 8 April 2026].

[5] P. Huoviala and P. F. Simmering, "Large Language Models for aspect-based sentiment analysis," arXiv, 2023.

[6] "Information Extraction in NLP," GeeksForGeeks, 9 January 2026. [Online]. Available: https://www.geeksforgeeks.org/nlp/information-extraction-in-nlp/. [Accessed 8 April 2026].

[7] "What is information extraction?," IBM, [Online]. Available: https://www.ibm.com/think/topics/information-extraction. [Accessed 8 April 2026].

[8] P. Pandey, "Can LangExtract Turn Messy Clinical Notes into Structured Data?," TowardsDataScience, 18 August 2025. [Online]. Available: https://towardsdatascience.com/can-langextract-turn-messy-clinical-notes-into-structured-data/. [Accessed 9 April 2026].

[9] GitHub, [Online]. Available: https://github.com/google/langextract. [Accessed 9 April 2026].

[10] A. Davis, "A Guide to Clustering Algorithms," Medium, 6 September 2024. [Online]. Available: https://medium.com/data-science/a-guide-to-clustering-algorithms-e28af85da0b7. [Accessed 12 April 2026].

[11] M. Mansurova, "Text Embeddings: Comprehensive Guide," Medium, 13 February 2024. [Online]. Available: https://medium.com/data-science/text-embeddings-comprehensive-guide-afd97fce8fb5. [Accessed 10 April 2026].

[12] "From Words to Vectors: Understanding Word2Vec in Natural Language Processing (NLP)," zilliz, [Online]. Available: https://zilliz.com/glossary/word2vec. [Accessed 10 April 2026].

[13] "Sentence Transformer," GeeksForGeeks, 23 July 2025. [Online]. Available: https://www.geeksforgeeks.org/nlp/sentence-transformer/. [Accessed 10 April 2026].

[14] Avantikaroy, "Agglomerative Clustering Explained: From Single Points to Meaningful Groups," Medium, 26 April 2025. [Online]. Available: https://medium.com/@avantika10roy/agglomerative-clustering-explained-from-single-points-to-meaningful-groups-c2a6e3d4bdf6. [Accessed 12 April 2026].

[15] C. P. Bathula, "Clustering Part 2: Everything About Hierarchical Clustering: Agglomerative vs Divisive Approaches," Medium, 25 June 2023. [Online]. Available: https://pub.towardsai.net/everything-to-know-about-hierarchical-clustering-agglomerative-clustering-divisive-clustering-badf31ae047. [Accessed 12 April 2026].

[16] P. Pai, "Hierarchical clustering explained," TowardsDataScience, 7 May 2021. [Online]. Available: https://towardsdatascience.com/hierarchical-clustering-explained-e59b13846da8/. [Accessed 12 April 2026].

[17] "Clustering Metrics in Machine Learning," GeeksForGeeks, 23 July 2025. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/clustering-metrics/. [Accessed 12 April 2026].

[18] "Amazon Reviews: Unlocked Mobile Phones," Kaggle, [Online]. Available: https://www.kaggle.com/datasets/PromptCloudHQ/amazon-reviews-unlocked-mobile-phones. [Accessed 13 April 2026].